dApp Connection Manual

For developers building decentralized applications on Synergy Network

Network: Synergy Testnet-beta
Chain ID: 338639
Revision: *2026-04-20*

Table of Contents

Part 0 — Before You Begin

0.1 Who this manual is for

This manual is written for developers who have never built a decentralized application (dApp) before and want to connect one to the Synergy Network. It assumes you can:

- Write basic JavaScript or TypeScript.
- Install and run Node.js (version 18 or later) on your own computer.
- Open a terminal and install npm packages.

It does not assume you know what an EVM chain, a JSON-RPC envelope, a Merkle Patricia tree, or a post-quantum signature is. Every concept is explained the first time it appears, and you can copy every example verbatim.

0.2 What a dApp actually is

A dApp ("decentralized application") is a normal web or mobile application that, instead of storing user data inside a private database your company owns, stores it inside a blockchain that thousands of independent computers operate together. The key differences from a normal web app:

- There is no central database you control. The database is the blockchain, and everyone can read it.
- Users do not log in with a password. They prove who they are by signing a cryptographic challenge with a key only they hold.
- Writes are permanent and cost a small fee, paid in the blockchain's native token. On Synergy that token is SNRG.
- Your frontend talks to the blockchain through a piece of software called an RPC node (here, Synergy's own Rust node).

0.3 What Synergy Network is, in plain words

Synergy is a proof-of-synergy blockchain built in Rust. It has three distinguishing features you will care about as a dApp builder:

- Post-Quantum Cryptography (PQC). Signatures, key-exchange, and account recovery use NIST-standardised post-quantum algorithms (ML-DSA, FN-DSA, ML-KEM). Normal ECDSA/secp256k1 wallets from Ethereum will not work on Synergy.
- Bech32m addresses with 37 purpose prefixes. Every address starts with a lowercase prefix that tells you what it is for: synw (user wallet), syns (smart contract), syna (authority), synu (SynID / "human-readable" account), synq (service), and so on.
- Native JSON-RPC over a custom TCP transport on port 5640. It looks like Ethereum's JSON-RPC but the method names are synergy_* and the request/response shapes are different.

Key fact There is no eth_* compatibility lane in the current Testnet-Beta node. Any tutorial you read that tells you to use MetaMask, ethers.js, or eth_sendRawTransaction against Synergy is wrong at this time. You will use synergy_* methods and the Synergy wallet.

0.4 What you will build by the end of this manual

By the time you reach the appendices you will have:

1. Connected to Testnet-Beta from a fresh terminal using nothing but curl.
2. Resolved your first SynID name and checked its balance.
3. Written a minimal JavaScript client class that wraps the Synergy JSON-RPC.
4. Simulated and submitted a signed transaction.
5. Deployed a smart contract written in Solidity against the Synergy node and called it from a React frontend.
6. Published a small dApp that asks the Synergy wallet extension for permission, reads an account, shows the user's SynID, and submits a vote to the Governance contract.

0.5 What you need installed

Tool	Version	Install command (macOS/Linux)
Node.js	*18.x or later*	`nvm install 20 && nvm use 20`
pnpm (optional)	*latest*	`npm install -g pnpm`
Foundry	*latest stable*	`curl -L https://foundry.paradigm.xyz | bash && foundryup`
Synergy wallet extension	*1.0.x*	`Load unpacked from wallet-app/apps/extension/dist`
A code editor	*anything*	VS Code, Cursor, Zed — whatever you use.

Part 1 — The Two Paths and Why This Manual Only Teaches One

1.1 The two connection stories you will hear

When you search the web for "how to connect a dApp to Synergy" you will find two competing stories:

Story A — the "EVM compatibility lane" story

This story says Synergy exposes an eth_* endpoint at a URL like testnet-evm-rpc.synergy-network.io, that you can register Synergy in MetaMask, that wagmi and viem will work out of the box, and that you can deploy unchanged Ethereum contracts.

Status as of Testnet-Beta (2026-04-20) This lane does not exist yet. There is no eth_* handler in the node code, no EVM interpreter, no secondary RPC router, and no DNS record for testnet-evm-rpc.*. Treat any tutorial relying on this lane as forward-looking; it describes a roadmap, not today's reality.

Story B — the "Synergy-native" story

This story says you talk to Synergy using synergy_* JSON-RPC methods on port 5640 (HTTPS in production), sign with ML-DSA or FN-DSA keys, use the Synergy wallet extension rather than MetaMask, and deploy contracts against Synergy's Rust runtime using Foundry pointed at the Synergy RPC URL.
This is the only path that currently works. Everything in this manual uses it.

1.2 Why Synergy is not "just Ethereum with a new name"

Three technical reasons, each of which changes what your dApp code must do:

7. Cryptography. Ethereum transactions are signed with a 65-byte secp256k1 signature. Synergy transactions are signed with a post-quantum signature between roughly 2,500 and 20,000 bytes depending on algorithm. Your wallet, your RPC request body, and your signing library all have to know which algorithm you used.
8. Addresses. Ethereum addresses are 20 bytes rendered as 0x-prefixed hex. Synergy addresses are SHA3-256 digests folded through Bech32m and rendered as lowercase strings like synw1q... that are 41 characters long and carry a prefix byte indicating account type.
9. Transaction envelope. Ethereum's EIP-1559 envelope is a 9-field RLP-encoded list. Synergy's envelope is a canonical JSON object with explicit field names (from, to, value, data, gasLimit, maxFee, nonce, chainId, signature, signatureAlgorithm) that must be serialised with JCS (RFC 8785) before signing.

1.3 Mental model to hold while reading the rest of this manual

Think of your dApp as three pieces of software that must all agree on the same envelope format:

- The frontend — your React/Vue/Svelte code in the browser, responsible for showing the user a screen and collecting their intent.
- The wallet — the Synergy extension, responsible for holding the user's keys and producing a post-quantum signature over the envelope.
- The node — the RPC endpoint (your own or a public one), responsible for validating the envelope, broadcasting it, and returning a transaction hash and receipt.

When any of the three drift from the others, your dApp breaks. Most of your bugs will be "field name mismatch" bugs. We will lean heavily on the canonical envelope in Part 4.

Part 2 — Network Configuration (Every Magic Number in One Table)

Copy these exactly. They are the most common cause of "it works locally but not on Testnet-Beta" confusion.

Key	Value on Testnet-Beta
Chain name	**Synergy Testnet-Beta**
Chain ID	**338639**
Chain ID (hex, for JSON-RPC)	`0x52b0f`
Native token symbol	**SNRG**
Native token decimals	**9**
Smallest unit name	**nWei (nano-Wei; 1 SNRG = 1,000,000,000 nWei)**
Block time (target)	**2.0 s**
HTTP JSON-RPC (public)	`https://testbeta-core-rpc.synergy-network.io`
WebSocket JSON-RPC (public)	`wss://testbeta-core-ws.synergy-network.io`
Wallet-API endpoint	`https://testbeta-wallet-api.synergy-network.io`
Cross-chain (SXCP) API	`https://testbeta-sxcp-api.synergy-network.io`
Analytics (Atlas) API	`https://testbeta-atlas-api.synergy-network.io`
Explorer	`https://testbeta-explorer.synergy-network.io`
Explorer REST docs	`https://testbeta-explorer.synergy-network.io/docs`
Local node HTTP port	`5640`
Local node WebSocket port	`5660`
P2P bootnode port	`5620`
Seed server port	`5621`
Faucet (self-serve)	**See docs/faucet-access.md once provisioned.**

2.1 Address prefixes you will actually see

Every account on Synergy is addressed by a Bech32m string. The first few characters ("human-readable part") tell you what the account is for. Here are the prefixes you are most likely to encounter while building a dApp:

Prefix	Purpose	Example (illustrative)
synw	**User wallet (end-user account that holds SNRG)**	`synw1qzr6h3x...7xqe4a`
syns	**Smart contract**	`syns1p8k9v2...3lqp7m`
syna	**Authority account (governance, oracle)**	`syna1c2k...`
synu	**SynID reverse-resolution target (human-readable)**	`synu1a4m...`
synq	**Service / relayer**	`synq1...`
sync	**Cluster validator set**	`sync1...`
syntxn	**Transaction hash (same-chain)**	`syntxn1blake3...`
synxxn	**Transaction hash (cross-chain)**	`synxxn1blake3...`

Why this matters If you try to send SNRG from a synw wallet to a syntxn address, the node rejects the transaction with error INVALID_ADDRESS_TYPE. Use the address engine (Part 6) to validate before building a transaction.

2.2 Adding Testnet-Beta to the Synergy wallet extension

On Testnet-Beta the network is pre-registered in the wallet extension. If you ever need to add it manually (for a custom build or a local devnet), use the "wallet_addNetwork" prompt with the following JSON:

```
{
  "chainId": "0x52b0f",
  "chainName": "Synergy Testnet-Beta",
  "nativeCurrency": {
    "name": "Synergy",
    "symbol": "SNRG",
    "decimals": 9
  },
  "rpcUrls": [
    "https://testbeta-core-rpc.synergy-network.io"
  ],
  "wsUrls": [
    "wss://testbeta-core-ws.synergy-network.io"
  ],
  "blockExplorerUrls": [
    "https://testbeta-explorer.synergy-network.io"
  ],
  "addressHrp": "synw"
}
```

Part 3 — Your First Round-Trip (Just curl and a Terminal)

Before touching React or TypeScript, confirm the network is reachable. Open a terminal and run:

```
curl -s -X POST https://testbeta-core-rpc.synergy-network.io \
  -H 'Content-Type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"synergy_blockNumber","params":[]}'
```

You should see a response like:

```
{"jsonrpc":"2.0","id":1,"result":"0x1c3"}
```

If you see a block number in hex, the network is reachable and your TLS chain is valid. If you get a TLS error, update curl; if you get a connection refused, check your own firewall first, then the status page.

3.1 Reading a balance

Pick any address from the explorer and ask the node for its balance:

```
curl -s -X POST https://testbeta-core-rpc.synergy-network.io \
  -H 'Content-Type: application/json' \
  --data
'{"jsonrpc":"2.0","id":1,"method":"synergy_getBalance","params":["synw1qzr6h3x...","latest"]}'
```

The return value is a string of SNRG in nWei (smallest unit). Divide by 1e9 to get human-readable SNRG.

3.2 A quick tour of the methods you will use most

Method	What it does
synergy_blockNumber	Latest block number (hex).
synergy_getBlockByNumber	Full block by number or tag (latest, pending, earliest).
synergy_getBalance	SNRG balance for a synw address, in nWei.
synergy_getTransactionCount	Nonce to use for this sender's next transaction.
synergy_getChainId	Chain ID as hex (expect 0x52b0f).
synergy_call	Read-only contract call; does not change state or cost gas.
synergy_estimateGas	Gas estimate for a tx envelope you plan to send.
synergy_sendTransaction	Broadcast a signed transaction envelope.
synergy_getTransactionByHash	Fetch a transaction by its syntxn1... hash.
synergy_getTransactionReceipt	Receipt (status, logs, gas used) for a confirmed tx.
synergy_getLogs	Event logs matching a filter.
synergy_getCode	Bytecode at a syns contract address.
synergy_getStorageAt	Raw storage slot at a contract.
synergy_getValidators	Current validator set.
synergy_getSynergyScoreBreakdown	Per-account Score, per category.

See the appendices Appendix A lists all 107 synergy_* methods with parameter types and example responses. Pin it.

Part 4 — The Synergy Transaction Envelope

Every state-changing request ("write") is a signed envelope. If you get nothing else from this manual, get the envelope right.

4.1 The canonical fields

Field	Type	Meaning
from	synw address	Sender account.
to	synw / syns / synu	Destination. A syns address means you are calling a contract.
value	string (nWei)	Amount of SNRG to transfer, in the smallest unit. Send "0" for a pure contract call.
data	hex string	Calldata. Empty ("0x") for a plain value transfer; otherwise an ABI-encoded function call.
gasLimit	string	Maximum gas units you will pay for.
maxFee	string (nWei/gas)	Maximum you will pay per gas unit. Fee model is EIP-1559-like.
nonce	string	Per-account sequence number. Get it with synergy_getTransactionCount.
chainId	string	"0x52b0f" on Testnet-Beta. Must be exact or the node rejects with CHAIN_MISMATCH.
signature	hex string	PQC signature over the JCS-canonicalised envelope (without this field).
signatureAlgorithm	string	One of ML-DSA-44, ML-DSA-65, ML-DSA-87, FN-DSA-512, FN-DSA-1024, SLH-DSA-128s, SLH-DSA-192s, SLH-DSA-256s.

4.2 How signing actually works (step by step)

1. Build the envelope with every field above except signature.
2. Canonicalise the JSON using JCS (RFC 8785). This means: sort keys lexicographically at every level, no extra whitespace, UTF-8 strings, and numeric forms as strings to avoid JS float issues.
3. Hash the canonical bytes with the hash your algorithm demands (e.g. SHAKE-256 for ML-DSA).
4. Sign with the user's PQC secret key. Keep the secret key in the wallet; never send it anywhere.
5. Append signature and signatureAlgorithm to the envelope.
6. Submit with synergy_sendTransaction.

4.3 Simulate before you sign

The spec requires wallets to call synergy_simulateTransaction and show the user decoded effects before any signing UI appears. Even though Testnet-Beta does not yet implement that RPC method, your dApp should already build the request shape so it works on Day 1 of rollout:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "synergy_simulateTransaction",
  "params": [
    {
      "from":  "synw1...",
      "to":    "syns1...",
      "value": "0",
      "data":  "0xa9059cbb000000000000...",
      "gasLimit": "0x30d40",
      "maxFee":   "0x2540be400",
      "nonce":    "0x3",
      "chainId":  "0x52b0f"
    },
    { "blockTag": "latest" }
  ]
}
```

The response, when implemented, returns a list of decoded effects (transfers, approvals, storage writes), a gas estimate, and any warnings. Show every warning in your UI; do not auto-sign if any simulator warning is present.

4.4 The five envelope mistakes you will make first

- Sending chainId as the decimal 338639 instead of the hex "0x52b0f". Always hex in JSON-RPC.
- Forgetting to canonicalise before hashing. JSON.stringify is not JCS; you must sort keys.
- Re-using a nonce. Call synergy_getTransactionCount right before signing.
- Mixing up "value" and "gasLimit" units. value is in nWei; gasLimit is in gas units (dimensionless).
- Putting leading zeroes in hex. "0x0000a" is wrong; use "0xa".

Part 5 — Your First Synergy Client (JavaScript, No Dependencies)

This is the smallest useful client. Save as synergy-client.js and import it anywhere.

```
// synergy-client.js
export class SynergyClient {
  constructor(rpcUrl = 'https://testbeta-core-rpc.synergy-network.io') {
    this.url = rpcUrl;
    this._id = 0;
  }

  async call(method, params = []) {
    const res = await fetch(this.url, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ jsonrpc: '2.0', id: ++this._id, method, params }),
    });
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const body = await res.json();
    if (body.error) {
      const err = new Error(body.error.message || 'RPC error');
      err.code = body.error.code;
      err.data = body.error.data;
      throw err;
    }
    return body.result;
  }

  // --- read helpers ---
  async blockNumber()            { return parseInt(await this.call('synergy_blockNumber'),
16); }
  async getBalance(addr)         { return BigInt(await this.call('synergy_getBalance', [addr,
'latest'])); }
  async getNonce(addr)           { return parseInt(await
this.call('synergy_getTransactionCount', [addr, 'latest']), 16); }
  async chainId()                { return await this.call('synergy_getChainId'); }
  async getReceipt(hash)         { return await this.call('synergy_getTransactionReceipt',
[hash]); }
  async callContract(tx)         { return await this.call('synergy_call', [tx, 'latest']); }

  // --- write (requires a pre-signed envelope) ---
  async sendTransaction(env)     { return await this.call('synergy_sendTransaction', [env]); }

  // --- event polling ---
  async getLogs(filter)          { return await this.call('synergy_getLogs', [filter]); }
}
```

Usage:

```
import { SynergyClient } from './synergy-client.js';
const s = new SynergyClient();
console.log('height', await s.blockNumber());
console.log('chain', await s.chainId());   // expect "0x52b0f"
console.log('bal',   await s.getBalance('synw1qzr6h3x...'));
```

5.1 JCS canonicalisation in 30 lines

Paste this into your project. It is enough for a hobby dApp; production code should use a battle-tested JCS library.

```
// jcs.js — minimal JSON Canonicalization Scheme
export function jcs(v) {
  if (v === null || typeof v !== 'object') return JSON.stringify(v);
  if (Array.isArray(v)) return '[' + v.map(jcs).join(',') + ']';
  const keys = Object.keys(v).sort();
  return '{' + keys.map(k => JSON.stringify(k) + ':' + jcs(v[k])).join(',') + '}';
}
```

Part 6 — Addresses, SynIDs, and the Address Engine

6.1 Why you cannot just accept any string

Addresses on Synergy are meaningful. The prefix tells the node what kind of account it is; the checksum tells it the address has not been mistyped. If your dApp hands the user's typed string straight to the wallet without validating it, the most likely outcome is that the user loses funds to a typo that passed a weak "starts with synw1" check.

6.2 Validating an address in JavaScript

Until the official npm package is published, use the reference Rust engine's logic transliterated into JS. This is a 40-line checksum verifier you can drop in:

```
// addr.js — SNTS-01 validator (abridged; production code should use @synergy-network/address)
const CHARSET = 'qpzry9x8gf2tvdw0s3jn54khce6mua7l';
const KNOWN =
['synw','syns','syna','synu','synq','sync','synv1','synv2','synv3','synv4','synv5',
               'synm','synl','synf','syne','syntxn','synxxn','synz'];

function bech32mPolymod(values) {
  const gen = [0x3b6a57b2, 0x26508e6d, 0x1ea119fa, 0x3d4233dd, 0x2a1462b3];
  let chk = 1;
  for (const v of values) {
    const b = chk >>> 25;
    chk = ((chk & 0x1ffffff) << 5) ^ v;
    for (let i = 0; i < 5; i++) if ((b >> i) & 1) chk ^= gen[i];
  }
  return chk ^ 0x2bc830a3; // bech32m constant
}

export function validate(addr) {
  if (addr !== addr.toLowerCase()) return 'MIXED_CASE';
  const sep = addr.lastIndexOf('1');
  if (sep < 1) return 'NO_SEPARATOR';
  const hrp = addr.slice(0, sep);
  const data = addr.slice(sep + 1);
  if (!KNOWN.includes(hrp)) return 'UNKNOWN_PREFIX';
  if (data.length < 6) return 'TOO_SHORT';
  const values = [];
  for (const c of data) {
    const i = CHARSET.indexOf(c);
    if (i === -1) return 'BAD_CHAR';
    values.push(i);
  }
  const hrpExp = [...hrp].map(c => c.charCodeAt(0) >> 5).concat(0).concat([...hrp].map(c =>
c.charCodeAt(0) & 31));
  if (bech32mPolymod([...hrpExp, ...values]) !== 0) return 'BAD_CHECKSUM';
  return 'OK';
}
```

6.3 Resolving a SynID

A SynID is a human-readable name registered in the Identity.sol contract. A dApp typically resolves a name to an address with synergy_resolveSynID and displays the reverse for confirmation with synergy_reverseResolveSynID. Until those convenience methods ship, you can call the registry contract directly:

```
// Resolve "alice" to an address via Identity.sol
const call = {
  to: '0x' /* Identity contract address from chain.json */,
  data: '0x' /* ABI-encoded resolve("alice") */
};
const result = await s.callContract(call);
// decode ABI-encoded address out of result
```

Naming rules SynID names are lowercase, 1-64 characters, character set [a-z0-9-]. You cannot register BobTheBuilder or bob.eth. Validate on the frontend before sending to the contract to avoid a wasted gas round-trip.

Part 7 — Smart Contracts: Deploying With Foundry

7.1 Why Foundry and what to configure

Synergy's contract format is Solidity 0.8.24, compiled to the same bytecode Foundry already knows how to produce. The only Synergy-specific setup is the RPC URL, the chain ID, and the signer (you cannot use a plain private key; you sign through the wallet).
Create a foundry.toml in your project:

```
[profile.default]
src = 'src'
out = 'out'
libs = ['lib']
solc = '0.8.24'
optimizer = true
optimizer_runs = 200

[rpc_endpoints]
synergy_testbeta = "https://testbeta-core-rpc.synergy-network.io"

[etherscan]
synergy_testbeta = { key = "${SYNERGY_EXPLORER_KEY}", url = "https://testbeta-
explorer.synergy-network.io/api" }
```

7.2 Writing a minimal vote contract

```
// src/Vote.sol
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

contract Vote {
    event Voted(address indexed voter, uint8 choice);
    mapping(address => uint8) public vote;

    function cast(uint8 choice) external {
        require(choice == 1 || choice == 2, "bad choice");
        vote[msg.sender] = choice;
        emit Voted(msg.sender, choice);
    }
}
```

7.3 Deploying through the wallet

You will not use forge create with a raw private key. Instead you will build the deployment transaction locally, hand it to the wallet for signing, and broadcast the signed envelope:

7. Run: forge build — produces out/Vote.sol/Vote.json with bytecode and ABI.
8. In your dApp, read the bytecode and wrap it into a Synergy transaction envelope whose "to" is omitted (contract creation) and "data" is the bytecode.
9. Call synergy_estimateGas against that envelope.
10. Ask the wallet to sign via window.synergy.request({ method: "synergy_signTransaction", params: [envelope] }).
11. Broadcast with window.synergy.request({ method: "synergy_sendTransaction", params: [signedEnvelope] }) — or your own client.
12. Wait for the receipt; the new contract's syns1... address is in receipt.contractAddress.

During Testnet-Beta Provider injection (window.synergy) is not yet in the wallet extension. Use Test-Beta's internal "Advanced" panel to sign the envelope and copy the signed JSON into your own fetch() that calls synergy_sendTransaction. The flow will move to window.synergy once the provider lands; your envelope code stays unchanged.

7.4 Calling the contract from your dApp

Reads are free and fast:

```
import { Interface } from 'ethers/lib/utils';
const iface = new Interface(abi);
const data = iface.encodeFunctionData('vote', [addr]);
const ret  = await s.callContract({ to: contractAddr, data });
const choice = iface.decodeFunctionResult('vote', ret)[0];
```

Writes go through the envelope + wallet flow in section 7.3.

Part 8 — Reading Events and Subscribing

8.1 Polling with synergy_getLogs

Until the WS subscription endpoint is live, poll every 2 seconds (one block time). Example: watch for every Voted event emitted by your contract:

```
const topic0 = '0x' + keccak('Voted(address,uint8)').slice(0,64);
let fromBlock = (await s.blockNumber());
setInterval(async () => {
  const latest = await s.blockNumber();
  if (latest <= fromBlock) return;
  const logs = await s.getLogs({ fromBlock, toBlock: latest, address: contractAddr, topics:
[topic0] });
  for (const log of logs) /* decode + display */;
  fromBlock = latest + 1;
}, 2000);
```

8.2 WebSocket subscriptions (when available)

Once synergy_subscribe is implemented on Testnet-Beta, replace polling with:

```
const ws = new WebSocket('wss://testbeta-core-ws.synergy-network.io');
ws.onopen = () => ws.send(JSON.stringify({
  jsonrpc: '2.0', id: 1, method: 'synergy_subscribe',
  params: ['logs', { address: contractAddr, topics: [topic0] }],
}));
ws.onmessage = (e) => {
  const msg = JSON.parse(e.data);
  if (msg.method === 'synergy_subscription') /* log in msg.params.result */;
};
```

Part 9 — Connecting the Synergy Wallet Extension

9.1 What the wallet will eventually expose

The Synergy wallet extension is a Manifest V3 Chrome/Firefox extension built from wallet-app/apps/extension. Once the provider-injection work described in the missing-features checklist lands, your dApp will see a global object window.synergy and an EIP-6963 announce event:

```
// Detect the wallet (EIP-6963 pattern)
window.addEventListener('eip6963:announceProvider', (e) => {
  const { info, provider } = e.detail;
  if (info.rdns === 'network.synergy.wallet') useProvider(provider);
});
window.dispatchEvent(new Event('eip6963:requestProvider'));
```

9.2 Requesting accounts

```
const accounts = await window.synergy.request({ method: 'synergy_requestAccounts' });
// => ['synw1...']
const chain = await window.synergy.request({ method: 'synergy_chainId' });
// => '0x52b0f'
```

The user sees a "Connect to example.com?" approval screen in the extension. Subsequent calls from the same origin reuse the granted session unless the user revokes it.

9.3 Signing a transaction

```
const envelope = { from: accounts[0], to: '...', value: '0', data: '0x...', gasLimit:
'0x30d40', maxFee: '0x2540be400', nonce: '0x3', chainId: '0x52b0f' };
const signed = await window.synergy.request({ method: 'synergy_signTransaction', params:
[envelope] });
const hash   = await window.synergy.request({ method: 'synergy_sendTransaction', params:
[signed] });
```

9.4 Designing your own approval UX

Even before provider injection lands, build your dApp as if the wallet were there. The Security Spec v1.3 requires wallets to:

- Show the simulated effects (decoded transfers, approvals, storage writes) before the Sign button.
- Recompute the destination address from the user's intent and warn on any mismatch with the envelope's "to".
- Reject any envelope with a type-0x04 delegation field.
- Show per-origin permission scope and expiry.

Your dApp should therefore never display "this transaction will transfer X" text of its own alongside a separate "Sign" button. Let the wallet own that screen.

Part 10 — Frontend Framework Notes

10.1 Where wagmi / viem / RainbowKit / Web3Modal stand today

All four libraries are Ethereum-first. They speak eth_*, they parse hex addresses as 20-byte blobs, and they expect secp256k1 signatures. None of them works against a Synergy node out of the box. You have three options:

- Build your own thin provider wrapper (recommended during Testnet-Beta).
- Wait for @synergy-network/wagmi-connector and @synergy-network/viem-transport to ship. They will translate eth_* calls into synergy_* under the hood and hide the difference from your UI code.
- Use viem's "custom" transport and pass it a lambda that rewrites method names and re-encodes parameters. This is a middle ground but defers the address-type, fee-model, and signature-bytes mismatches to your own code.

10.2 Minimal React provider pattern

```
// SynergyProvider.jsx
import { createContext, useContext, useEffect, useState } from 'react';
import { SynergyClient } from './synergy-client.js';
const Ctx = createContext(null);
export function SynergyProvider({ children }) {
  const [client] = useState(() => new SynergyClient());
  const [account, setAccount] = useState(null);
  const [chainId, setChainId] = useState(null);
  async function connect() {
    if (!window.synergy) throw new Error('Synergy wallet not detected');
    const [a] = await window.synergy.request({ method: 'synergy_requestAccounts' });
    setAccount(a);
    setChainId(await window.synergy.request({ method: 'synergy_chainId' }));
  }
  return <Ctx.Provider value={{ client, account, chainId, connect
}}>{children}</Ctx.Provider>;
}
export const useSynergy = () => useContext(Ctx);
```

10.3 Reading balances in a component

```
function Balance() {
  const { client, account } = useSynergy();
  const [bal, setBal] = useState(null);
  useEffect(() => {
    if (!account) return;
    const id = setInterval(async () => setBal(await client.getBalance(account)), 2000);
    return () => clearInterval(id);
  }, [account]);
  if (!account) return <button onClick={connect}>Connect Synergy Wallet</button>;
  return <div>{String(bal)} nwei</div>;
}
```

Part 11 — Synergy-Specific Features Your dApp Can Use

11.1 Score

Every account has a Score — a composite reputation number derived from on-chain activity (staking longevity, delegation stability, uptime if validator, governance participation, etc.). Scores are observable: synergy_getSynergyScoreBreakdown returns the categories that built a given score.
If your dApp wants to gate access or weight votes by Score, read it from the node and surface it; do not attempt to compute it yourself.

11.2 Staking

Staking happens in the Staking.sol genesis contract. Your dApp does not need to embed the ABI until you explicitly want to let users stake from inside it. When you do, the flow is: (1) read available validators via synergy_getValidators, (2) read delegation details via synergy_getDelegators(validatorAddress), (3) send a staking transaction through your wallet.

11.3 Governance

The Governance contract implements proposal creation, voting, and execution. A simple "vote on proposal N" flow from a dApp is:

13. Read the proposal list from the contract.
14. Render a simple Yes / No / Abstain UI.
15. Build an envelope calling vote(proposalId, choice).
16. Sign + submit through the wallet.
17. Subscribe to the Voted event for live tallies.

11.4 Cross-chain (SXCP) transfers

SXCP is Synergy’s cross-chain transfer protocol. A dApp sends a transfer to a synxxn address; relayers attest to it; it settles on the destination chain. Status fields to surface: pending, attested, finalised. Use the SXCP API endpoint listed in Part 2.

11.5 Post-quantum signer selection

Most user flows should use FN-DSA-1024 (fast signing, compact enough). Large-value transfers and validator operations should use ML-DSA-87 (larger signature, stronger parameters). Your dApp does not choose the algorithm — the wallet does. Your job is only to display the algorithm tag that comes back in the signed envelope so a power user can verify it.

Part 12 — Testing, Debugging, and Going to Production

12.1 Local devnet

Clone synergy-testnet-beta and follow setup-guide.md. The node listens on localhost:5640 with the same JSON-RPC surface. Point your dApp at http://localhost:5640 and test full end-to-end.

12.2 Debugging envelope errors

Error code	Meaning and fix
-32601	Method not found. Check spelling; verify method exists in Part 3 table.
-32602	Invalid params. Diff your request body against the envelope schema in Part 4.
-32010	RECIPIENT_TAMPERED — the wallet's re-derived destination disagrees with your envelope's "to". Ensure your UI shows the same address the envelope carries.
-32011	INSUFFICIENT_FUNDS — usually nwei vs SNRG unit confusion.
-32012	SCORE_TOO_LOW — the target contract requires a higher Score than the sender has.
-32013	AUTHORIZATION_EXPIRED — signed envelope's expiry is in the past. Refresh and re-sign.
-32015	CHAIN_MISMATCH — chainId in envelope doesn't match the node.
-32020	NONCE_GAP — another tx already used this nonce, or you skipped one.
-32030	SIGNATURE_INVALID — most likely JCS canonicalisation is wrong.

12.3 Going to production checklist

- Hard-pin the RPC URL and chain ID in your app config; do not derive them from the wallet.
- Re-validate every address the user types. Never trust "starts with synw" alone.
- Show the simulation warnings from the wallet verbatim.
- Log, but do not display, PQC signature bytes.
- For any transfer over a threshold, require the wallet's timelock tier (see wallet-core/src/policy/timelock.js).
- Rate-limit client-side calls to the public RPC; it is shared.
- Ship a status banner that reads your own infra's health from status.synergy-network.io.

Appendix A — Complete synergy_* Method Reference (abbreviated)

This appendix lists the method name and a one-line purpose. For full parameter and return shapes, see docs/api-reference.md once the port-number correction lands (it currently says 8545; the correct port is 5640).

Method	Purpose
synergy_blockNumber	Latest block number.
synergy_getBlockNumber	Alias of above (deprecated).
synergy_getBlockByNumber	Block by number or tag.
synergy_getBlockByHash	Block by hash.
synergy_getLatestBlock	Latest block object.
synergy_sendTransaction	Broadcast signed envelope.
synergy_getTransactionPool	Current mempool snapshot.
synergy_registerRelayer	Register as SXCP relayer (authority).
synergy_unregisterRelayer	Remove SXCP relayer.
synergy_relayerHeartbeat	SXCP liveness ping.
synergy_getRelayerSet	Active relayer set.
synergy_getRelayerHealth	Relayer health metrics.
synergy_getSxcpStatus	Cross-chain bridge status.
synergy_submitAttestation	Relayer attestation of a foreign event.
synergy_getEventAttestation	Fetch an attestation by id.
synergy_getAttestations	List attestations by filter.
synergy_slashRelayer	Governance slashing of relayer.
synergy_nodeInfo	Node software + chain info.
synergy_getDeterminismDigest	Hash over deterministic state.
synergy_getValidators	Current validator set.
synergy_getValidator	Single validator by address.
synergy_getTokenBalance	Non-native token balance.
synergy_getTokens	List registered tokens.
synergy_createWallet	Internal: create wallet server-side.
synergy_getWallet	Internal: fetch wallet.
synergy_getAllWallets	Internal: enumerate wallets.
synergy_signTransaction	Sign an envelope using a server-held key (dev-only).
synergy_sendTokens	High-level token transfer.
synergy_stakeTokens	Stake to a validator.

Method	Purpose
synergy_stakeTokensDirect	Self-stake.
synergy_unstakeTokens	Begin unbonding.
synergy_getStakedBalance	Staked amount for address.
synergy_getStakingInfo	Per-account staking summary.
synergy_registerValidator	Apply to become validator.
synergy_approveValidator	Governance approval.
synergy_getTopValidators	Top-N by stake.
synergy_slashValidator	Governance slash.
synergy_getBlockRange	Many blocks at once.
synergy_getTransactionByHash	Tx by hash.
synergy_getTransactionsInBlock	All txs in a block.
synergy_getValidatorStats	Validator performance stats.
synergy_getTokenStats	Per-token usage stats.
synergy_getNetworkStats	Global network metrics.
synergy_createToken	Create a native-framework token.
synergy_mintTokens	Mint to holder.
synergy_burnTokens	Burn from holder.
synergy_transferTokens	Token transfer.
synergy_getAllBalances	Multi-token balance view for an address.
synergy_getTransferHistory	Per-address transfer history.
synergy_getNodeStatus	Node health and peer count.
synergy_getSyncStatus	Sync progress.
synergy_getBlockValidationStatus	Per-block validation state.
synergy_getValidatorActivity	Validator activity feed.
synergy_getSynergyScoreBreakdown	Score per category for an address.
synergy_getPeerInfo	P2P peer detail.
synergy_getTransactionReceipt	Receipt by tx hash.
synergy_getTransactionCount	Nonce / tx count.
synergy_getBalance	SNRG balance.
synergy_gasPrice	Current base gas price (placeholder).
synergy_call	Read-only call.
synergy_estimateGas	Gas estimate.
synergy_getLogs	Filtered event logs.
synergy_getCode	Bytecode at address.

Method	Purpose
synergy_getStorageAt	Raw storage slot.
synergy_getBlockTransactionCount	Tx count in block.
synergy_getBlockReceipts	All receipts in a block.
synergy_getPendingTransactions	Mempool (public view).
synergy_getTransactionByBlockNumberAndIndex	Indexed access.
synergy_getTransactionByBlockHashAndIndex	Indexed access.
synergy_maxFeePerGas	Current max fee (placeholder).
synergy_maxPriorityFeePerGas	Current priority fee (placeholder).
synergy_getFeeHistory	Fee history (placeholder).
synergy_getChainId	Chain id (currently decimal — treat with care).
synergy_getValidatorByCluster	Validator by cluster id.
synergy_getValidatorRewards	Rewards due.
synergy_getValidatorPerformance	Uptime + score.
synergy_getValidatorQueue	Activation queue.
synergy_requestValidatorExit	Voluntary exit.
synergy_getValidatorSlashingHistory	History of slashes.
synergy_getClusterInfo	Cluster metadata.
synergy_getClusterRewards	Cluster-level rewards.
synergy_proposeClusterChange	Propose cluster change.
synergy_getStakingRewards	Per-account reward accrual.
synergy_getStakingAPY	Current APY.
synergy_getDelegatedStakes	Delegations from account.
synergy_getDelegators	Delegators to a validator.
synergy_claimRewards	Claim pending rewards.
synergy_getRewardsProjection	Projected rewards.
synergy_getUnstakingPeriod	Unbonding period in blocks.
synergy_status	High-level node status.

Appendix B — Quick Reference Card

Tear this out and pin it by your monitor.

Thing	Value / rule
Chain ID	338639 decimal / 0x52b0f hex
Token	SNRG (9 decimals, smallest = nwei)
RPC URL	https://testbeta-core-rpc.synergy-network.io
Block time	2 s
Address example	synw1qzr6h3x...
Bad address example	SynW1... (uppercase invalid)
Hash example	syntxn1blake3...
Nonce source	synergy_getTransactionCount
Signing	JCS + PQC, never secp256k1
Gas	EIP-1559-style: gasLimit + maxFee
Canonicalise before hash	Always
Simulate before sign	Always (when available)
Re-validate recipient	Always
Ethereum compatibility lane	Not live on Testnet-Beta

Appendix C — Glossary

Term	Definition
JSON-RPC	Remote-procedure-call protocol over HTTP or WebSocket where requests and responses are JSON objects. Synergy uses JSON-RPC 2.0.
Envelope	The full object describing a transaction's intent (from, to, value, data, gas, nonce, chainId) together with its signature.
JCS	JSON Canonicalization Scheme (RFC 8785). A way to serialise a JSON object to bytes deterministically so signatures are reproducible.
Nonce	Per-account counter that prevents replay. Increases by 1 per accepted transaction.
Bech32m	Encoding used by Synergy addresses. Includes a checksum and a prefix that identifies the account type.
PQC	Post-Quantum Cryptography. Algorithms believed resistant to large-scale quantum attackers.
ML-DSA / FN-DSA / SLH-DSA	Digital-signature algorithms Synergy accepts. ML and FN are lattice-based; SLH is hash-based.
ML-KEM	Post-quantum key-encapsulation mechanism used by Synergy's wallet for recovery capsules.
SynID	Human-readable name registered in the Identity.sol contract; resolves to a synw / synu address.
SXCP	Synergy Cross-Chain Protocol. Relayer-attested transfers across chains.
SCETP	Same-Chain External Transfer Protocol. Cross-environment transfers within Synergy.
Score	Composite on-chain reputation number derived from activity; returned by synergy_getSynergyScoreBreakdown.
Exposure profile	Security-spec term for which RPC methods are reachable at a given endpoint (Public read, Authenticated client, Authority plane, Operator, WS sub).